

Automation of pair programming practice through LLM

Advisor: Shahid Hussain –Assistant Professor, Computer Science and Software Engineering
Kareen Chhangani

CONTEXT

- Two developers work together at one workstation
- One writes code; the other reviews and guides in real time
- Improves code quality and reduces defects
- Enhances problem-solving and knowledge sharing
- Resource-intensive (two developers for one task)
- Difficult to scale across large teams
- Scheduling and coordination reduce productivity
- Effectiveness depends on partner compatibility
- Challenging for remote and distributed teams
- Need for scalable, lower-overhead collaborative alternatives

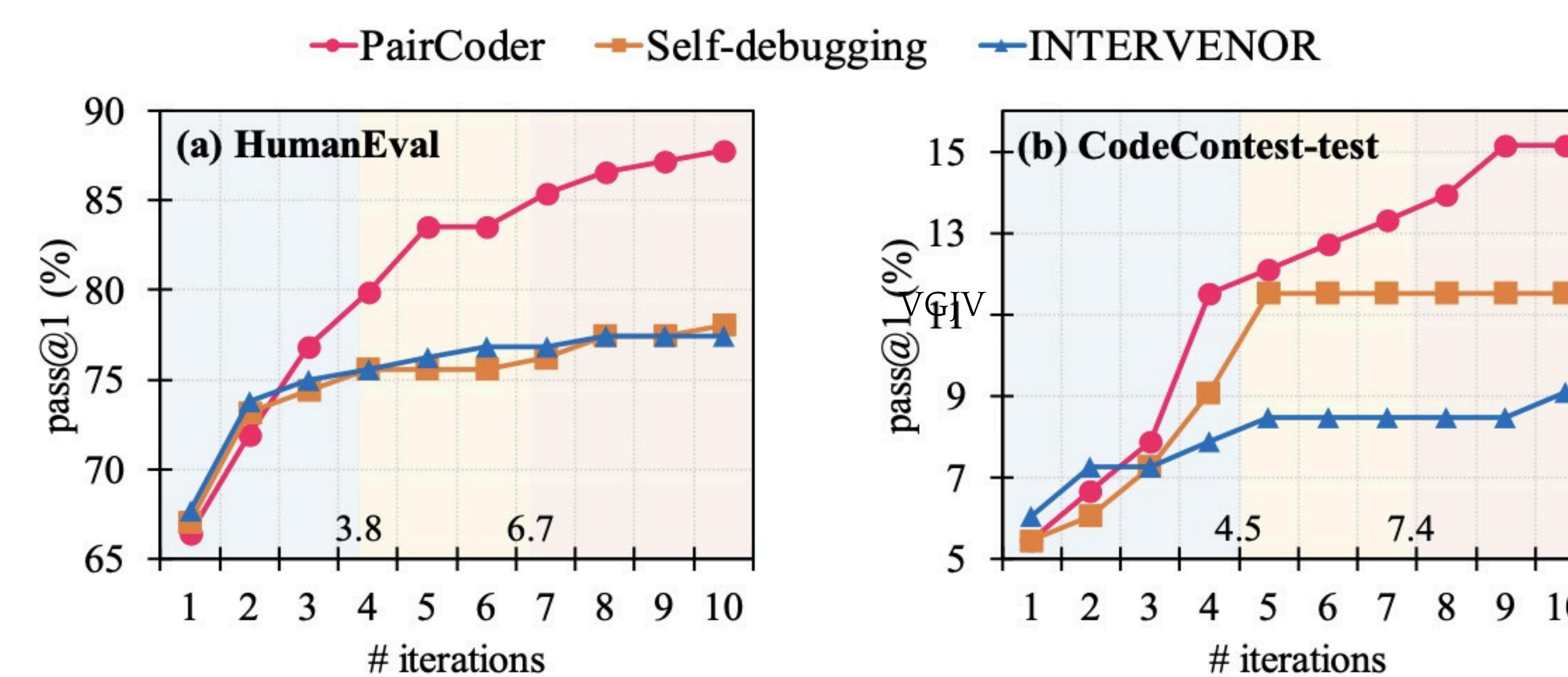
METHOD

- Structured collaboration approach inspired by pair programming without continuous co-location
- Clearly defined, alternating Driver and Navigator–style roles to promote shared understanding and accountability
- Use of collaborative tools supporting real-time and asynchronous interaction for distributed teams
- Application to real development tasks with observation of workflow efficiency and communication
- Evaluation of whether pair programming benefits can be preserved while improving scalability and flexibility

PSEUDOCODE

```

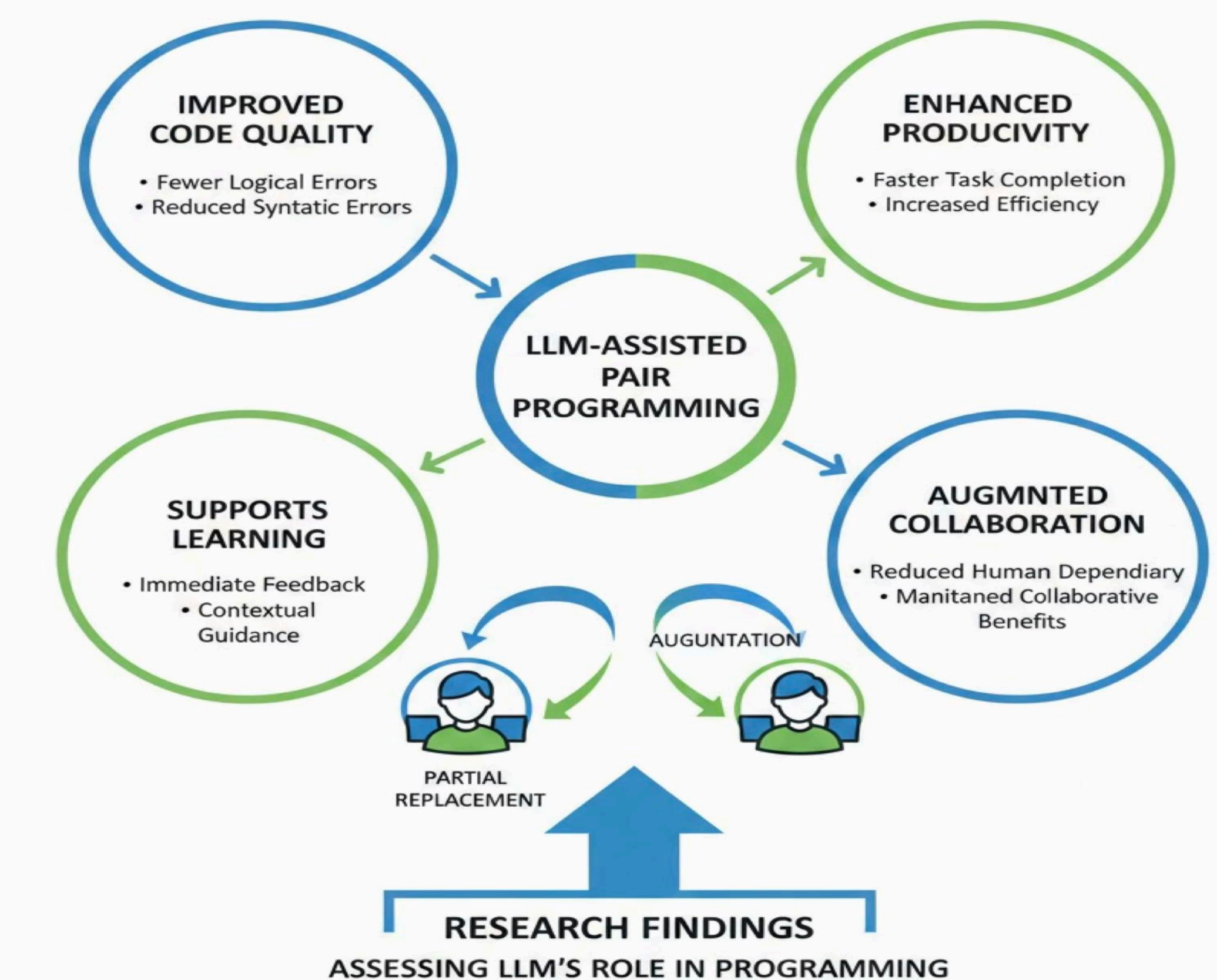
FOR each task DO
  Assign human Driver, provide task context
  WHILE task not completed DO
    Driver writes/updates code
    LLM_Navigator reviews code and suggests improvements
    Driver applies selected suggestions
  ENDWHILE
  Store final code and log interaction
ENDFOR
  
```



This figure illustrates the performance of various coding frameworks across multiple iterations on benchmarks.

OBSERVATION

- The graph shows AI Navigator–Driver setups improve solution quality over iterations, while other approaches plateau early.
- An LLM as Navigator provides continuous feedback and knowledge sharing
- Maintains learning and team collaboration without sacrificing productivity



This figure illustrates how the Pair Programming enhances LLM-assisted pair programming

CONCLUSION

- An LLM can act as the Navigator in pair programming, providing continuous feedback and knowledge sharing
- Automating code review, suggestions, and best-practice guidance reduces the need for two human developers
- Supports scalable, flexible, and distributed collaboration for modern software teams.

REFERENCES

Zhang, H., Cheng, W., Wu, Y., & Hu, W. (2024). A Pair Programming Framework for Code Generation via Multi-Plan Exploration and Feedback-Driven Refinement. In 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24). <https://doi.org/10.1145/3691620.3695506>